

Mathilda vs Mathematica — Evaluator Throughput

Mathilda — evaluator throughput

Contents

Mathilda vs Mathematica — Evaluator Throughput	1
Results	1
Reading of the results	4
Method	4
Improvements applied (2026-08-11)	5
What this points the work at next	7
Follow-up: Apply top-level fast path + Plus/Times short-circuit (2026-08-12)	7
Symbol table and expression hashing (2026-08-11)	9
Extended stress tests (2026-08-12)	10
Appendix A — Benchmark source code	11

Mathilda vs Mathematica — Evaluator Throughput

A head-to-head of Mathilda's recursive expression-tree evaluator against Wolfram Mathematica 14.0.0 (wolframscript, macOS x86-64), across seven categories of complex stress tests that exercise the generic evaluation loop, the pattern matcher, and the rule engine — and nothing else. This is deliberately not a test of `Compile[]` or of the packed-array machine kernels: every workload is symbolic, Rational, or scalar-recursive so that both engines stay on their general evaluators (see Method → Evaluator-only proof).

It is the companion to [MATHILDA_PATTERN_MATCHER_PERFORMANCE.md](#), which established that Mathilda's matcher is at parity or $4\times$ – $135,000\times$ faster than Mathematica, and flagged the one open evaluator-side gap — the per-attempt constant factor ("Case B", $1.6\times$ slower). This document asks whether that constant is a one-off or systemic. It is systemic.

Headline. Two opposite results, cleanly separated by what they stress:

1. Backtracking / matching: Mathilda wins decisively — $44\times$ to $236\times$ faster. Its shortest-first enumeration and "last blank consumes the remainder" pruning turn searches Mathematica brute-forces into polynomial ones.
2. Generic per-node evaluation throughput: Mathilda is $\sim 1.6\times$ – $4.3\times$ slower, median $1.89\times$, uniformly across dispatch, rewrite, arithmetic, structural, and control-flow workloads. This is the same per-step constant factor "Case B" isolated, now shown to dominate the whole evaluator.

The matcher is a solved problem; the interpreter constant is where the work is. That is a well-scoped, cross-cutting engineering target, not an algorithm gap — Mathilda and Mathematica run the identical evaluation semantics here.

Results

The two Mathilda columns bracket the 2026-08-11 optimisation campaign: pre is the pre-campaign baseline; post is Mathilda after the six shipped improvements (analysed under Improvements applied below). The pre/post

ratio columns are Mathilda ÷ WL 14.0 in that snapshot (>1 = Mathilda slower; <1 = Mathilda faster). WL 14.0 is the fixed reference — the campaign touched only Mathilda — and its own timing moved between the two runs only as measurement noise (within a few percent, larger only on the sub-0.1 ms micro-cases where the absolute swing is microseconds), so one WL 14.0 column serves both. Timings are AbsoluteTiming, minimum of 3 runs after one warm-up (heavy cases: one timed run), identical .m source and identical hard-coded inputs on both engines, value-checked case-by-case (every check agreed — 0 disagreements). Net effect, counted against the 1.5× parity line used in Improvements applied: 17 slower / 11 ahead → 8 slower / 20 ahead.

Where Mathilda trailed pre-campaign: generic evaluation throughput

Category	Case	Mathilda pre	Mathilda post	WL 14.0	pre	post
rewrite	pairwise fold //. , Range[800]	16.8 ms	8.2 ms	3.9 ms	4.3×	2.1×
rewrite	pairwise fold //. , Range[400]	4.6 ms	2.7 ms	1.1 ms	4.1×	2.4×
arithmetic	Case B, untyped __ sequence + length guard	10.3 ms	5.8 ms	2.6 ms	4.0×	2.3×
arithmetic	Length[Plus @@ 2000 symbolic terms] (Orderless sort)	5.0 ms	4.6 ms	1.5 ms	3.3×	3.1× (→ 2.0×, 2026-08- 12)
higher- order	Apply[Plus, ..., {1}] (@@@ MapApply), 5000 triples — 2026-08-12†	11.5 ms	7.4 ms	4.68 ms	2.5×	1.6×
higher- order	Map /@ over 5000 compound g[a,b,c] — 2026-08-12†	4.19 ms	0.52 ms	0.66 ms	6.4×	0.80× (faster)
higher- order	Scan over 5000 compound g[a,b,c] — 2026-08-12†	0.90 ms	0.54 ms	0.71 ms	1.3×	0.76× (faster)
higher- order	MapIndexed over 5000 compound g[a,b,c] — 2026-08-12†	5.55 ms	1.57 ms	1.35 ms	4.1×	1.2×
dispatch	ack[3,4] nested recursion	19.1 ms	9.5 ms	6.2 ms	3.1×	1.5×
dispatch	mutual recursion, depth 800	1.1 ms	0.56 ms	0.36 ms	3.0×	1.6×
dispatch	fib[28] naive recursion	1.57 s	0.85 s	585 ms	2.7×	1.5×
structural	Fold[fp, 0, 5000 f[k] nodes]	5.4 ms	3.1 ms	2.3 ms	2.3×	1.3×
construction	ReplaceAll rebuild, 15000 nodes	19.2 ms	8.3 ms	8.7 ms	2.2×	0.96×
structural	Position[4000 nodes, f[_?EvenQ]]	3.2 ms	1.7 ms	1.5 ms	2.1×	1.1×
structural	Map[sq, 5000 f[k] nodes]	5.3 ms	5.3 ms	2.5 ms	2.1×	2.1×
dispatch	Nest[incr, 0, 50000] (DownValue per step)	36.6 ms	18.8 ms	18.1 ms	2.0×	0.98×
control- flow	Collatz col[27], guarded DownValues	0.248 ms	0.165 ms	0.129 ms	1.9×	1.3×
arithmetic	Total[Table[k/3, 20000]] (Rational)	21.6 ms	20.5 ms	11.4 ms	1.9×	1.7×
rewrite	bubble sort //. , 30 elts	7.1 ms	4.7 ms	3.8 ms	1.9×	1.1×
structural	Count[5000 nodes, f[_?PrimeQ]]	2.5 ms	1.7 ms	1.6 ms	1.6×	1.1×
arithmetic	Case B typed (Repeated[_Integer]) + length guard	13.0 ms	9.8 ms	8.3 ms	1.6×	1.2×

Case B typed reproduces the matcher-doc figure exactly (1.6×). The rest place that number in context: the same constant factor is present everywhere ordinary Length/Plus/Times/Equal, list rebuilds, and DownValue dispatch happen.

† The @@@ MapApply row's Mathilda pre/post bracket the 2026-08-12 Apply follow-up (not the 2026-08-11 campaign the other rows measure): pre is the value recorded for this benchmark at Mathilda 0.039 (experiment 49 below), post is after the specialised top-level Apply fast path, the apply_child past-max short-circuit that makes @@@ cheap, and the Plus/Times short-circuits — see Follow-up: Apply top-level fast path ... below. WL 14.0 is unchanged. @@@ / Apply[... , {1}] was 2.47× (11.5 ms) and is now 1.6× (7.4 ms); its residual, like the Orderless-sum row, is Table construction + the per-node symbolic-Plus evaluation, not the Apply restructuring.

‡ The Map/Scan/MapIndexed rows are the depth-independence wins, measured directly against WolframKernel 14.0 (experiment 52, Apply / Map / Scan / MapIndexed restructuring vs WolframKernel 14.0 below, run identically on both engines): pre is pre-fix Mathilda over 5000 compound g[a, b, c] elements — where the cost grew with element depth because map_at_level/scan_at_level/mi_at_level descended into every element's whole subtree — and post is after the past-max refcount short-circuit. Map/Scan now run faster than WL 14.0 (/@ 0.80×, Scan 0.76×). MapIndexed is likewise depth-independent (4.1× → 1.2×) but stays just above WL, because it builds a {position} vector and a two-argument wrapper per element — construction WL's native kernel does a little more cheaply, and the only thing left after the descent is gone. The deeper the elements, the larger the win (at depth 3, /@ was 7.1 ms pre-fix → 0.52 ms; MapIndexed 8.4 ms → 1.6 ms).

Where Mathilda led pre-campaign: backtracking, and a few throughput cases

Category	Case	Mathilda pre	Mathilda post	WL 14.0	pre	post
backtracking	Case M equal-sum blocks, N=80	22.8 ms	10.9 ms	5.38 s	236× faster	504× faster
backtracking	Case M equal-sum blocks, N=60	10.7 ms	5.9 ms	1.56 s	146× faster	265× faster
backtracking	Case D palindrome, 200 ints	3.7 ms	1.8 ms	163 ms	44× faster	89× faster
construction	LeafCount over 60001 leaves	0.075 ms	0.076 ms	0.322 ms	4.3× faster	4.2× faster
control-flow	FixedPoint Newton $\sqrt{2}$ on Rationals	0.039 ms	0.037 ms	0.111 ms	2.8× faster	2.1× faster
construction	Depth[Nest[gg, x0, 900]]	0.076 ms	0.077 ms	0.128 ms	1.7× faster	1.7× faster
control-flow	Do[If[PrimeQ[k], ...], 10000]	3.6 ms	3.5 ms	4.4 ms	1.2× faster	1.3× faster
arithmetic	Nest[#+a&, x, 4000] then a->0	5.0 ms	5.1 ms	4.9 ms	~1.0× (tie)	~1.0× (tie)
control-flow	Do[s = s + a, 100000]	86.0 ms	86.2 ms	72.9 ms	1.18× slower	1.21× slower
construction	build 15000 node[k, k^2, k^3]	13.9 ms	12.4 ms	10.8 ms	1.28× slower	1.17× slower
rewrite	Peano add //. , 60+60	1.5 ms	0.48 ms	1.1 ms	1.31× slower	2.3× faster

Reading of the results

The backtracking wins (Case D, Case M) are the matcher doc, generalized

Case M is a contiguous equal-sum partition — a polynomial problem that looks exponential. Mathilda evaluates each guard the instant a boundary is proposed and lets the trailing blank absorb the remainder, so infeasible prefixes die immediately; Mathematica explores every boundary before its guards reject it. The gap widens with size (146× at N=60, 236× at N=80) exactly because WL's work grows geometrically while Mathilda's stays polynomial. Case D (nested palindromic sequence blanks) is the same mechanism at 44×. These confirm, on a fresh deterministic corpus, the matcher doc's central finding.

The throughput gap is real, uniform, and constant-factor

Every "slower" row is bound by the per-step evaluator constant, not by any algorithm difference — the two engines do the identical work in the identical order:

- Dispatch (fib, Ackermann, mutual, **Nest**): 2.0×–3.1×. A trivial body (`fib[n-1]+fib[n-2]`) means the row measures almost nothing but the cost of one `DownValue` call: pattern lookup, `MatchEnv` allocation, argument evaluation, and rebuilding the result node. `fib[28]` is the cleanest single number: 1.57 s vs 585 ms.
- Rewrite (`//.` folds): the worst rows at 4.1×–4.3×. Each pass matches cheaply, binds the whole tail to `r___`, and rebuilds the list by splicing the sequence — $O(n)$ list reassembly per pass, $O(n^2)$ total. This is the per-step node/args reassembly cost, amplified.
- Arithmetic (Case B, Orderless sum, Rational total): 1.6×–3.3×. Building $\{x\}, \{y\}, \{z\}$ from bound sequences and evaluating `Length/Plus/Times/Equal` per attempt; the untyped-vs-typed Case B delta (4.0× vs 1.6×) shows the per-element `_Integer` head re-check costs a further chunk.
- Structural (**Map/Fold/Count/Position**): 1.6×–2.3×. The generic per-element dispatch loop over symbolic `f[k]` nodes.

The consistency is the finding. A median of 1.89× across five unrelated categories says the cost is in the shared machinery — `evaluate_step`'s per-node reassembly (`src/eval.c`, a fresh `expr_new_function` every pass even when nothing changed), `MatchEnv` allocation (`src/match.c`), and the per-attempt guard/Length/arithmetic evaluation — not in any one head.

Where Mathilda already ties or wins on throughput tells us where the cost is not

- **Do[If[PrimeQ[k], ..., 10000]** (1.2× faster) and Newton on Rationals (2.8× faster): here the per-iteration body is dominated by a heavy builtin (`PrimeQ`, Together on growing rationals), so the interpreter constant is a small fraction and Mathilda's fast primitives win. The constant only hurts when the body is trivial.
- **Nest[#+a&, x, 4000]** collapse (tie) and **Do[s=s+a]** (1.18×): symbolic `Plus` accumulation is near parity — `Plus/Times` canonicalize internally and Mathilda's is competitive.
- **LeafCount / Depth** traversal (1.7×–4.3× faster): Mathilda's $O(1)$ refcount `expr_copy` and copy-on-write sharing make whole-tree traversal cheap. Traversal is a Mathilda strength; rebuild (`ReplaceAll`, 2.2× slower) is not, because it allocates.

This bracketing points squarely at allocation + per-node reassembly on trivial bodies as the lever, which is exactly what the improvement plan targets.

Method

- Engine. Mathilda 0.037 (`-O3 -std=c99`, GCC 16.1.0, GMP 6.3.0, MPFR 4.2.2, FLINT 3.6.0, Accelerate). Reference. Mathematica 14.0.0 for Mac OS X x86 (64-bit), `wolframscript 1.11.0, -file`. Host. macOS 15.7.4, x86-64.
- Harness. `benchmarks/run_all.py` (`make bench-gap`) runs the same `.m` in Mathilda `-file` and `wolframscript -file`, one process per experiment per system, `cwd` set to the experiment folder. Timing

is `AbsoluteTiming` (never `Timing[]`, which sums CPU over threads), minimum of 3 runs after one warm-up; heavy cases (Case M) use one timed run.

- Correctness gate. Every case emits `check[label, value]`; a case whose two engines disagree is reported CHECK-FAIL and its timing discarded. All 28 cases agreed.
- Fairness. Mathematica auto-packs numeric lists and auto-compiles numeric `Table/Do` bodies, so a numeric-array workload would measure its compiler. Every workload here uses symbolic (`f[k]`, `c[k]` `x`), Rational (`k/3`), or scalar-recursion data, holding both engines on their generic evaluators — never by disabling a fast path on one side only.
- Syntax note. Mathilda's parser does not accept the postfix `../...` (`Repeated/RepeatedNull`) operators; the `Repeated[...]` long form and `___/___/` all work and are what the patterns use. This is a minor parser gap, not an evaluator one — the semantics are identical on both engines.

Evaluator-only proof (no packing, no compilation)

The whole point of the exercise is that these numbers are the evaluator. Two independent checks confirm it:

1. Pack-gate diagnostic (`MATHILDA_PACK_DIAG=gate`). Categories 1, 5, 6, 7 materialize zero packed buffers. Categories 2, 3, 4 materialize a buffer exactly once per input, at the point a packed integer list is handed to an unaware matcher head — `MatchQ` (1000 elts), `ReplaceList` (460 elts), `ReplaceRepeated` (6150 elts) — element counts equal to input-size $\times$ reps, i.e. the one-time $O(n)$ input handoff, not the per-attempt backtracking work.
2. **MATHILDA_NO_PACK=1** timing invariance. Disabling packing entirely leaves the measured times unchanged — in fact slightly faster (Case B 13.1 $\rightarrow$ 11.6 ms, fold400 5.2 $\rightarrow$ 3.9 ms, fold800 17.1 $\rightarrow$ 14.4 ms), because packing an input that is immediately materialized is pure overhead. Packing therefore never accelerates any measured work; if anything the default numbers slightly understate Mathilda on the matcher-input rows.

No workload calls `Compile[]` or a numeric `Table/Do` body WL would auto-compile, so the no-compilation constraint holds by construction.

The corpus

Seven experiments under `benchmarks/40-eval-dispatch ... benchmarks/46-eval-controlflow`, 28 cases total, each an `.m-only` folder using the shared `bench/check` harness. The full source of every case is reproduced in [Appendix A](#). Reproduce with:

```
python3 benchmarks/run_all.py --only 40,41,42,43,44,45,46 --system mathilda,wolfram
```

Improvements applied (2026-08-11)

Six levers from the analysis shipped; a seventh was built, measured, and rejected. Cumulatively the suite moved from 17 slower / 11 ahead to 8 slower / 20 ahead with no regressions; the table below is the net baseline $\rightarrow$ final effect (ratios are Mathilda $\div$ WL 14.0).

Case	baseline	final	Mathilda baseline $\rightarrow$ final
<code>// . fold, Range[800]</code>	4.3 $\times$	2.1 $\times$	16.8 $\rightarrow$ 8.2 ms
<code>// . fold, Range[400]</code>	4.1 $\times$	2.4 $\times$	4.6 $\rightarrow$ 2.7 ms
Case B untyped	4.0 $\times$	2.3 $\times$	10.3 $\rightarrow$ 5.8 ms
<code>ack[3,4]</code>	3.1 $\times$	1.5 $\times$	19.1 $\rightarrow$ 9.5 ms
Orderless sum (<code>Plus @@ 2000 c[k]</code> <code>x</code>)	3.3 $\times$	3.1 $\times$	5.0 $\rightarrow$ 4.6 ms
mutual recursion 800	3.0 $\times$	1.6 $\times$	1.1 $\rightarrow$ 0.56 ms
<code>fib[28]</code>	2.7 $\times$	1.5 $\times$	1.57 s $\rightarrow$ 0.85 s
bubble sort <code>// .</code>	1.9 $\times$	parity (1.1 $\times$)	7.1 $\rightarrow$ 4.7 ms

Case	baseline	final	Mathilda baseline → final
Nest[incr] / Count / Position / Case B typed / ReplaceAll rebuild	1.6–2.5×	parity	crossed the 1.5× line (ReplaceAll rebuild now 0.96×, Mathilda ahead)

Improvement 1 — pooled **MatchEnv** (src/match.c). `apply_down_values_def` (src/symtab.c) allocated and freed one **MatchEnv** — three mallocs — per candidate **DownValue**, so recursive rewriting churns millions (`fib[28]` ≈ 2.5M). `env_new/env_free` now recycle the whole struct including its `symbols/values` arrays through a bounded static free-list, making reuse allocation-free after warm-up. External contract unchanged; per-binding cleanup preserved; pooled envs are program-lifetime “still reachable”. `bench_match` shows it directly: Cases 333→196 ns/elem, Position 1083→583, ReplaceAll 390→237 (1.4×–1.9×). Attacks the dispatch and matcher rows.

Improvement 2 — machine-integer fast path for **Plus/Times** (src/plus.c, src/times.c). Both ran several $O(n)$ scans (NDArray probe, neg-Plus, SeriesData, inexact contagion, then grouping) before combining terms. A fast path after the `n==0/1` checks folds an all-EXPR_INTEGER argument list with the overflow-checked `ci_add_i64/ci_mul_i64` and returns a single integer; overflow (or any non-int operand) falls through to the generic bigint path, so results are unchanged (`9223372036854775807+1` and `3037000500`² still give exact bigints, matched against WL byte-for-byte). Attacks the `Plus[int, int]` that dominates `fib` and integer loops — the largest single move (`fib[28]` 1.33 s → 1.06 s on top of the pool).

Improvement 3 — borrow **MatchEnv** keys (src/match.c). The pool recycled the struct and arrays but `env_set` still `strdup`'d each binding's key and `env_free` freed it — one malloc + free per binding, millions in `fib`/structural mapping. Every caller passes a key that already outlives the per-attempt env (an interned symbol name, or the “\$OptionsPattern\$” literal), so the key is now stored by borrow — no `strdup`, never freed — with a pointer-first comparison (interned keys hit immediately) and a `strcmp` fallback for the cross-TU literal callers. Further trims the binding path: `fib[28]` 1.06 → 1.05 s, Case B typed into parity, `bench_match` ReplaceAll 237 → 187 ns/elem; valgrind dropped to 420 blocks (two below baseline — fewer allocations).

Improvement 4 — skip **evaluate()** for atomic arguments (src/eval.c `evaluate_step`). Profiling the `//.` fold (sample) put the whole cost in `eval_and_free` re-evaluating the result list each pass — the per-element `evaluate()` calls in the argument loop, not `replace_bindings` or the match (which the earlier reassembly guess had wrongly assumed). A raw atom (number/string/BigInt/MPFR/NDArray/Compiled) always evaluates to itself, so `evaluate()` there only `expr_copys` it after paying the recursion-depth, trace, deadline-check and fixed-point-loop overhead per call. The loop now recognises an atomic argument and goes straight to `expr_copy` (symbols excluded — they may carry `OwnValues`). Provably equivalent, and the single biggest move on the folds: `fold800` 4.3× → 2.2×, `fold400` → 2.3×, and it also cut Case-B-untyped (3.1× → 2.5×, whose guard builds `{x}, {y}, {z}` integer lists) and dropped the ReplaceAll-rebuild row off the slow list.

Improvement 5 — skip the redundant sort in **builtin_plus** (src/plus.c). Profiling `Plus @@ Table[c[k] x, {k, 2000}]` put ~half the time in the final-term `qsort` and its `expr_compare` per comparison (the grouping hash table and reconstruction were already $O(1)/O(n)$). But a `Plus` built in order arrives canonically ordered (`c[1] x < c[2] x < ...`), and every re-evaluation of an already-sorted sum re-sorts terms that never moved — pervasive, since sums flow through the evaluator to a fixed point. An $O(n)$ already-sorted pre-check now guards the `qsort` (mirrors the Orderless fast path in `evaluate_step`); it only sorts on the first descent. Re-evaluating a canonical 2000-term sum 1.89 → 1.12 ms (–40%); `expr_compare` samples 307 → 80, `qsort` 61 → 20. The Orderless-sum row itself moved 3.3× → 3.0× — its residual is now Table construction and the $O(n)$ Infinity/contagion pre-scans, not the sort.

Improvement 6 — borrow subset/remainder slices in ordered sequence matching (src/match.c). Re-profiling the fold after Improvement 4 showed the cost had moved off `eval_and_free` and into the matcher: `match_args_internal` allocated and copied a fresh remainder array (via `extract_subset`) at every pattern element — one `{a_, b_, r___}` pass paid ~4 $O(n)$ tail copies. But `next_combination` is gated on `is_orderless`, so for an ordered head the do-while runs once with `comb = [0..k-1]`: the consumed run is the contiguous prefix and the remainder is the contiguous suffix `exprs+k`. Both are slices of the caller's array and are read-only, so borrow them instead of `malloc+extract_subset` (the `comb` index array is skipped

too). A general matcher speedup: every match over an ordered head's arguments now avoids the per-element subset/remainder/comb allocation. `bench_match`: `ReplaceAll` 187 → 99.8 ns/element, Cases 216 → 126, Position 634 → 500. Fold Range[800] 9.2 → 8.2 ms, bubble sort // . 7.6 → 3.8 ms (2×), Case D 3.8 → 1.6 ms, `ack[3,4]` 13 → 9.5 ms, `fib[28]` into parity. The 298-case `match_stress_corpus` passes, and `valgrind` is clean over both the borrow (ordered) and copy (Orderless) paths.

Rejected — adopt the per-step args array (`expr_new_function_adapt` in `evaluate_step`, to skip the second `malloc+memcpy` when a large `List/Plus` is rebuilt). Built and A/B-measured: `fold800` 0.7%, `bigsum4k` ~1%, only `rebuild20k` and `fold600` ~7% — within noise, because the large-node cases are dominated by the operation (rule application in `replace_bindings`, Orderless grouping), not the reassembly copy. Not worth a new public API + a change to the hottest function for a sub-noise win (Simplicity First / Minimal Impact). Reverted.

Verified for all six shipped improvements: the 298-case `match_stress_corpus` passes (0 failed, 0 crashed); `bench_eval/bench_match` gates PASS; matcher correctness (nonlinear patterns, `OptionsPattern`, guards, Cases/`ReplaceAll`, `ReplaceList`, `OrderlessPatternSequence`) and arithmetic (`Expand[(1+x+y)^12]`, term collection, canonical ordering) match WL byte-for-byte; all 28 suite value checks agree across engines (0 CHECK-FAIL); the `ctest` failures (`zero_test`, `moebiusmu/primenu` number-theory tests — flaky ~2/8 on unmodified `de6a69f`, `bench_pack/bench_compile` timing-gates flaky even idle, and the SVD/eigen/lapack SEGFAULTs) are all pre-existing/flaky; `valgrind` stays at or below the 422-block macOS baseline over both the borrow (ordered) and copy (Orderless) matcher paths, with no invalid access.

What this points the work at next

Every clear hotspot the profiles surfaced has been taken: `DownValue`-dispatch allocation, integer arithmetic, `MatchEnv` keys, atomic-argument re-evaluation, the Orderless sort, and the ordered-sequence remainder copy. 20 of 28 rows are now at or ahead of Mathematica; the 8 that remain are general per-node constant factor with no single hotspot — the profiles spread across `Table/list` construction of many symbolic terms, the $O(n)$ Infinity/contagion pre-scans in `builtin_plus`, and `expr_hash/expr_compare` per term. In priority order:

1. Symbolic **Table**/list construction at scale — the Orderless-sum row (3.1×) is now dominated by building the 2000 symbolic terms, not the sum; the same generic list-construction constant underlies the residual fold and Map rows.
2. **builtin_plus** per-arg pre-scans — the $O(n)$ Infinity/Indeterminate classification and inexact-contagion passes run over every term of a large sum even when none is special; a combined single pass would trim the first-time grouping.
3. Per-element blank head re-checks (`src/match.c blank_head_matches`) — the residual typed-vs-untyped Case B delta.

These are diminishing-returns, cross-cutting constant-factor work: the six shipped changes took the worst outliers from 4.3× to ~2.3× and moved nine rows across the 1.5× line into parity, and each further step is a broader, riskier change to the shared evaluator core for a smaller marginal gain.

`tests/bench_eval.c` already gates several of these in-process; the flagship rows (`fib`, Case B) should be added so a regression trips the gate directly.

Follow-up: Apply top-level fast path + Plus/Times short-circuit (2026-08-12)

Priority 1 above — the arithmetic sentinel `Length[Plus @@ Table[c[k] x, {k, 2000}]]`, the last row still trailing WL at 3.1× — was profiled down to its three critical-path operations (`Table` build, `@@`, the `Plus` grouping) and taken from 4.69 → 2.95 ms (3.1× → 2.0×). Two avoidable costs were removed; one was a finding the campaign above had missed.

- **Apply** did redundant work (`src/funcprog.c`). For the default top-level spec `f @@ list` called `get_depth` — a full recursive traversal of the whole 2000-element tree — at every node (its only consumer is the negative-level branch a non-negative spec never takes) and then structurally cloned every element subtree (~4000 interior nodes) though `Expr` trees are immutable and `refcount-shared`. A specialised fast path in `builtin_apply` (mirroring the existing `f @@ <|assoc|>` case) now builds `f[elements...]` by `refcount-sharing` each element and evaluates once; `apply_at_level` skips

get_depth and refcount-shares past spec.max for any non-negative spec, so @@@/explicit-level Apply benefit too. Isolated f @@ list: ~1.3 ms → ~0.05 ms. For @@@ (Apply[... , {1}]) the general path still recursed once per element and once per sub-element — the level-2 calls doing nothing but refcount-share — so an apply_child helper folds the past-max case into an O(1) share instead of a recursive call. This is why the @@@ MapApply row in the opening table moved 2.47× → 1.6× (11.5 → 7.4 ms on the 5000-triple benchmark); isolated f @@@ list restructuring dropped ~15% (0.93 → 0.78 ms over 5000 elements). MapApply is a common operation, so this is a broad win, not just a benchmark one.

- **Map (/@)** had the same overhead — worse (src/funcprog.c map_at_level). Map is bottom-up and always descended to the leaves, calling get_depth at every node and rebuilding every interior node, so the common f /@ list cost grew with element depth though f only touches level 1. The identical fix (skip get_depth for non-negative specs; once past the max level share the subtree with an O(1) refcount bump rather than descend + rebuild) makes it depth-independent: f /@ 5000 g[a,b,c] 4.19 → 0.56 ms (−87%), f /@ 5000 g[h[...],h[...]] 7.12 → 0.57 ms (−92%). /@ is one of the most common operations in the language, so this is the broadest win of the three.
- **Plus/Times** ran five full pre-scans that all find nothing for an ordinary symbolic sum/product (src/plus.c, src/times.c). A single fused detection pass now short-circuits straight to grouping unless a factor is NDArray / neg-Plus / SeriesData / inexact / Infinity — a provably-correct superset of the passes' triggers; the passes themselves are unchanged.

Both are correctness-neutral (new tests/test_core_algebra.c pins every touched path against WL Full-Form; the 298-case matcher corpus and the arithmetic/collect suites are unchanged; valgrind is identical to the empty-session baseline). The three flagship bench_eval rows (fib, Case B, Plus-2000) called for above are now armed.

The residual 2.0× is **Table**, not **Plus/Apply**. Of the 2.95 ms, 2.43 ms is now Table's per-element evaluation of a symbolic Times — the systemic per-node constant this document is about, which alone exceeds WL's 1.5 ms total. Its single largest lever is the fixed-point double pass: builtin_times/builtin_plus unconditionally rebuild their result and trip the change flag (eval.c §3.4 comment), so every element pays a second confirm-stable evaluate_step. Removing it means making the arithmetic builtins report accurate no-change status, or a cheap already-canonical check — a core-evaluator change of broad blast radius, left for a separate measured effort rather than chased on one synthetic row (the same diminishing-returns calculus the campaign reached above).

Apply / Map / Scan / MapIndexed restructuring vs WolframKernel 14.0 — dedicated test cases

These are the head-to-head test cases for the 2026-08-12 fast paths, measured against WolframKernel 14.0 run directly (/Applications/Mathematica.app/Contents/MacOS/WolframKernel -noprompt -script; same WL 14.0 as the rest of the document, just the raw kernel rather than wolframscript). The identical .m source ([benchmarks/52-eval-apply-map/apply_map.m](#), reproduced in [Appendix A.13](#)) runs on both engines. Every operand is symbolic (ff/gg/hh/aa/... all undefined) so both stay on their general evaluators, and each list is built once, outside the timing so the case measures only the Apply/Map/Scan/MapIndexed restructuring — not Table. Timing is the minimum of 5 AbsoluteTiming reps, best of three process runs, in integer microseconds; ratio is Mathilda ÷ WL (<1 = Mathilda faster). All eleven paired value-checks agree across engines (0 disagreements), so correctness is verified directly against WL 14.0.

Operation (5000 elements)	Mathilda	WK 14.0	ratio
ff @@ list — Apply, level 0	0.088 ms	0.225 ms	0.39× (2.6× faster)
ff @@@ list — MapApply, level 1	0.836 ms	1.244 ms	0.67× (1.5× faster)
ff /@ list — Map, compound elts (depth 2)	0.524 ms	0.658 ms	0.80×
ff /@ list — Map, deep elts (depth 3)	0.516 ms	0.614 ms	0.84×
Map[ff, list, {0}] — Map, level 0	<0.001 ms	<0.001 ms	O(1) both
Scan[ff, list] — Scan, compound (depth 2)	0.542 ms	0.714 ms	0.76× (1.3× faster)
Scan[ff, list] — Scan, deep (depth 3)	0.535 ms	0.733 ms	0.73× (1.4× faster)
MapIndexed[ff, list] — compound (depth 2)	1.573 ms	1.353 ms	1.16×

Operation (5000 elements)	Mathilda	WK 14.0	ratio
MapIndexed[ff, list] — deep (depth 3)	1.596 ms	1.295 ms	1.23×
Apply[Plus, list, {1}] — MapApply Plus	2.225 ms	1.999 ms	1.11×
Length[Plus @@ list] — Plus grouping	1.301 ms	0.808 ms	1.61×

Two clean readings:

- The restructuring itself now beats Wolfram. Pure Apply/Map/Scan head-rewrites and walks — where the fast paths recount-share the untouched substructure instead of cloning or descending into it — are 1.2×–2.6× faster than WL 14.0, @@ most of all (a single evaluate over recount-shared arguments). Map/Scan over deep elements are the sharpest structural cases: the pre-fix code grew with element depth (Map 7.1 ms, Scan 1.4 ms at depth 3), and both now land near 0.5 ms — flat in depth, below WL. MapIndexed got the same short-circuit and is likewise depth-independent now (compound 5.5 → 1.6 ms, deep 8.4 → 1.6 ms), but it stays ~1.2× WL: it does strictly more per element than Map — it builds a {position} vector and a two-argument f [part, pos] wrapper — and that construction constant, not any descent, is what remains.
- The two rows still above 1.0× are symbolic **Plus**, not the Apply/Map. Apply [Plus, ..., {1}] (1.11×) evaluates 5000 three-term symbolic sums, and Plus @@ (1.61×) canonicalises+groups a 5000-term sum — both bound by the per-node symbolic-arithmetic constant this document isolates, exactly as the Length [Plus @@ 2000 ...] sentinel is. The Apply/Map work took those heads off the restructuring critical path; what remains is the shared Plus/Times evaluator constant.

Symbol table and expression hashing (2026-08-11)

Follow-up to the question "is the symbol (hash) table or the expression hash function part of the per-node constant?" Two findings, opposite verdicts:

- **Symbol table** — not a contributor. Name→definition resolution is already an O(1) cached-pointer read: each EXPR_SYMBOL node stores its resolved SymbolDef* in data.symbol.def on first touch (eval.c), attributes are read once as a bitmask, and the resolved hdef is threaded through DownValue, builtin and kernel dispatch. Steady-state evaluation does zero string hashes per node; the djb2/65535-bucket table is hit only on first-touch of a freshly-constructed symbol. No change warranted.
- **expr_hash** — was uncached, now memoized. It was FNV-1a recomputed by walking the whole subtree on every call, and is called per-element in Plus/Times grouping, Association key indexing, and the set ops. A lazily-filled Expr.hash_cache field (see the changelog entry for the invalidation contract and the MATHILDA_HASH_VERIFY correctness gate) makes a repeated hash of the same subtree O(1). Measured up to ~14× on dedup/Tally of repeated deep structural elements (4.07→0.30 ms) and ~1.3–1.6× on repeated Association lookups; results unchanged, no new leaks, full ctest clean but for the documented pre-existing failures.

The eval rows above (dispatch/rewrite/arithmetic/structural) are not hash-bound — each rep builds fresh nodes (the inner Table churns the eval clock), so the cache does not carry across reps and those rows are unchanged. The cache pays off where the same node is hashed many times: Associations, set ops, and re-grouping a stable sum whose term nodes persist.

- **expr_compare** — attempted, measured a wash, reverted. Re-grouping a stable symbolic sum is dominated not by hashing but by expr_compare's polynomial-degree pass (collect_symbols_in + expr_poly_degree per pairwise comparison, sort.c), which builtin_plus's already-sorted check runs O(n) times per re-grouping. A toggle that skips step 3 entirely put its ceiling at ~38% of the re-group cost (sum 460→282 ms). Two ways to bank it:

1. Fuse step 3 into one multi-variable degree walk (build_degmap) instead of collect + one expr_poly_degree per symbol — safe (no per-node state, no canonical-order risk, cross-checked against the per-symbol oracle under a MATHILDA_SORT_VERIFY build). Built and A/B-measured: a wash on the re-group rows (460→461 ms) and only a noisy ~3–10% on Sort. It must build both complete degree maps before comparing, losing the per-symbol version's early-exit at the first

differing symbol; for the tied terms that dominate re-grouping, both do full work. Reverted (Simplicity First, like Improvement 7 above).

2. A persistent per-node degree-signature cache (compute each node's signature once, reuse across the $O(n)$ check and every re-grouping) is the only thing that banks the ceiling — but it needs +8–16 bytes/node, allocation, and a second canonical-order invalidation surface (a stale signature is a silent wrong-order bug in `Sort/OrderedQ/display`). Disproportionate risk for a $\sim 1.4\times$ gain on a synthetic repeated-regroup row; not pursued.

Extended stress tests (2026-08-12)

A second battery of 19 cases in five experiments (`benchmarks/47-eval-memoization ... benchmarks/51-eval-arithmetic-extreme`) pushes the suite past its calibration sizes and into four evaluator regimes the first seven did not isolate: memoization and `DownValue`-table growth, lexical scoping (`Module/With`/pure functions), higher-order restructuring (`Outer/Sort/FoldList/Apply`), and extreme-size dispatch/rewrite/arithmetic. Measured on Mathilda 0.039 (the post-campaign binary plus the structural-hash memoization) against the same WL 14.0 reference, identical `.m` source on both engines, one process per case per engine; all 19 value checks agree across engines (0 CHECK-FAIL). Ratio is Mathilda ÷ WL 14.0 (>1 = Mathilda slower). Full source is in [Appendix A](#) (§A.8–A.12); reproduce with `python3 benchmarks/run_all.py --only 47,48,49,50,51 --system mathilda,wolfram`.

Every case here keeps both engines on their general evaluators. The deliberate exclusion is symbolic differentiation, which was built, measured, and cut: WL's `D[]` is a native C kernel that returns in microseconds and then caches the result (a warm-up run makes the timed reps read $\sim 1\ \mu\text{s}$), so a D-based row would pit Mathilda's interpreted `deriv.m` rewrite rules against a compiled builtin — exactly the one-sided fast path the Method forbids. `Module/With/Fold`/memoization have no such kernel; both engines interpret them identically.

Category	Case	Mathilda	WL 14.0	Ratio
memoization	<code>memo fib[1000]</code> (1000 stored <code>DownValues</code>)	44.1 ms	2.46 ms	17.9×
memoization	<code>memo binom[100,50]</code> (2-arg, ~ 2500 entries)	493 ms	7.33 ms	67×
memoization	build 5000 facts <code>f[k]=k^2</code> , then sum	945 ms	7.91 ms	119×
scoping	<code>Module</code> recursion <code>gs[1500]</code>	7.2 ms	4.1 ms	1.76×
scoping	<code>With</code> substitution loop, 30000	21.6 ms	31.7 ms	1.5× faster
scoping	<code>Module</code> generation, 30000 scopes	92.1 ms	94.3 ms	$\sim 1.0\times$ (tie)
scoping	pure-function <code>Fold</code> , 6000 deep	4.2 ms	4.0 ms	1.06×
higher-order	<code>Outer</code> , 150×150 symbolic	5.1 ms	4.0 ms	1.26×
higher-order	<code>Sort</code> 3000 symbolic terms	4.8 ms	1.12 ms	4.3×
higher-order	<code>FoldList</code> , 6000 symbolic	2.8 ms	2.3 ms	1.21×
higher-order	<code>Apply[Plus, ..., {1}]</code> , 5000 triples	11.5 ms → 7.4 ms	4.68 ms	2.47× → 1.6× (2026-08-12)
dispatch-extreme	<code>naive fib[31]</code>	3.60 s	2.43 s	1.48×
dispatch-extreme	<code>ack[3,6]</code> nested recursion	168 ms	106 ms	1.58×
dispatch-extreme	<code>pairwise fold //. , Range[1200]</code>	20.3 ms	8.3 ms	2.44×
dispatch-extreme	Case M equal-sum blocks, $N=90$	15.5 ms	8.87 s	572× faster
arithmetic-extreme	<code>Length[Plus @@ 5000 symbolic terms]</code>	11.8 ms	4.48 ms	2.64×
arithmetic-extreme	<code>Do[s=s+a, 300000]</code>	257 ms	227 ms	1.13×

Category	Case	Mathilda	WL 14.0	Ratio
arithmetic-extreme	Total[Table[k/3, 60000]] (Rational)	61.4 ms	34.9 ms	1.76×
arithmetic-extreme	Fold[Times, 1, Range[12000]] = 12000!	32.0 ms	10.3 ms	3.11×

Three readings:

- The per-node constant holds at scale. Scoping, higher-order, and the extreme-size dispatch/rewrite/arithmetic rows land in the same 1.1×–3.1× band as the main suite — the campaign's constant factor is stable as sizes grow, and Mathilda even leads on With-loop substitution (1.5×) and ties on Module generation. Sort (4.3×) is the one outlier, bound by `expr_compare`'s polynomial-degree pass — the same cost the Symbol table and expression hashing section measured — not the generic loop.
- The backtracking margin widens with size. Case M at N=90 is 572× faster — up from 236× at N=80 in the main suite — because WL's search grows geometrically (8.87 s) while Mathilda's fail-fast pruning holds it at 15.5 ms.
- DownValue-table insertion is $O(n^2)$ — a new, verified target. The memoization rows are the largest gaps in either document: 18× → 67× → 119× as the table grows to 1000 → ~2500 → 5000 entries. The build-5000-facts row is almost pure insertion (5000 Sets, then a linear sum), and a doubling test pins it: building N facts costs 66 → 235 → 937 → 3731 ms at N = 1250 → 2500 → 5000 → 10000 — a ~4× step per doubling, i.e. $O(n^2)$. Mathilda's per-entry cost rises with table size (44 → ~190 μs/entry) while WL's stays flat (~2 μs/entry), which places an $O(n)$ DownValue insertion in `src/symtab.c` against WL's amortized $O(1)$. This is not the per-attempt constant the main suite isolates; it is an algorithmic scaling gap in rule storage, and the sharpest optimization lever these tests surface.

Appendix A — Benchmark source code

Every experiment is a single `.m` file, loaded and run unmodified by both Mathilda `-file` and `wolframscript -file`; there is no per-engine variant. That is the whole point of the method (see Method → Evaluator-only proof): the two columns cannot be timing two different programs, because they run the identical source over identical hard-coded inputs. The listings below are therefore the exact code both engines executed. Each file opens with `Get["../harness.m"]`; `run_all.py` sets the working directory to the experiment folder so that relative `Get` resolves (Mathilda has no `$InputFileName`).

Every timed case is paired with a `check[...]` that emits a single-line value; `run_all.py` compares those across engines and, on any disagreement, reports CHECK–FAIL and discards the timing. All 28 checks of the main suite agreed, as did all 19 of the Extended stress tests (§A.8–A.12) — 47 in total, 0 disagreements.

A.0 Shared harness (`benchmarks/harness.m`)

The reporting helpers used by every experiment. `bench` runs one untimed warm-up, then reports the minimum of `$BenchReps = 3 AbsoluteTiming` runs (minimum, not mean: noise on a loaded machine can only add). `benchOnce` is the single-run, no-warm-up form for a case whose one run is already seconds (Case M at N=80). `check` prints `InputForm` so the value is guaranteed single-line ASCII that round-trips identically in both systems. Only the helpers these seven experiments use are shown; the full file carries additional coverage/portability shims.

```
$BenchReps = 3;
```

```
SetAttributes[bench, HoldRest];
bench[label_String, expr_] := Module[{ts},
  expr; (* warm-up, untimed *)
  ts = Table[First[AbsoluteTiming[expr]], {$BenchReps}];
  Print["BENCH\t", label, "\t", ToString[Round[1000. Min[ts], 0.001]]];
```

```
];

SetAttributes[benchOnce, HoldRest];
benchOnce[label_String, expr_] :=
  Print["BENCH\t", label, "\t",
    ToString[Round[1000. First[AbsoluteTiming[expr]], 0.001]]];

check[label_String, value_] :=
  Print["CHECK\t", label, "\t", ToString[value, InputForm]];
```

A.1 Experiment 40 — dispatch (**40-eval-dispatch/eval_dispatch.m**)

Recursive DownValue dispatch — per-call pattern lookup, MatchEnv allocation, and per-step node reassembly — on scalar recursion / a scalar loop that never builds a list, so nothing is diverted to the packed-array kernels or Compile[].

```
Get["../harness.m"];

(* ---- 1. Naive Fibonacci: exponential DownValue dispatch ---- *)
Clear[fib]; fib[0] = 0; fib[1] = 1; fib[n_] := fib[n - 1] + fib[n - 2];
bench["fib[28] naive recursion", fib[28]];
check["fib[28] naive recursion", fib[28]];

(* ---- 2. Ackermann: deeply nested dispatch ---- *)
Clear[ack]; ack[0, n_] := n + 1; ack[m_, 0] := ack[m - 1, 1];
ack[m_, n_] := ack[m - 1, ack[m, n - 1]];
bench["ack[3,4] nested recursion", ack[3, 4]];
check["ack[3,4] nested recursion", ack[3, 4]];

(* ---- 3. Mutual recursion: linear dispatch depth ---- *)
Clear[evenR, oddR]; evenR[0] = True; oddR[0] = False;
evenR[n_] := oddR[n - 1]; oddR[n_] := evenR[n - 1];
bench["mutual recursion depth 800", evenR[800]];
check["mutual recursion depth 800", evenR[800]];

(* ---- 4. Nest over a DownValue: tight rewrite loop ---- *)
(* Nest is iterative (no C-stack recursion); each step is one DownValue match
+ one Plus. Scalar integer accumulator, never a list. *)
Clear[incr]; incr[x_] := x + 1;
bench["Nest[incr, 0, 50000]", Nest[incr, 0, 50000]];
check["Nest[incr, 0, 50000]", Nest[incr, 0, 50000]];
```

A.2 Experiment 41 — backtracking (**41-eval-backtracking/eval_backtracking.m**)

The structural matcher's backtracking throughput on flat-list sequence patterns with guards — enumeration order, fail-fast guards, and the "last sequence blank consumes the remainder" pruning rule. Inputs are deterministic Table[Mod [...]], identical in both engines.

```
Get["../harness.m"];

(* ---- 1. Case D: palindromic sequence blanks (perf-doc D) ---- *)
L200 = Table[Mod[k, 7] + 1, {k, 200}];
bench["Case D palindrome, 200 ints",
  MatchQ[L200, {___, a___, b___, c___, b___, a___, ___} /; Length[{c}] > 10]];
check["Case D palindrome, 200 ints",
  MatchQ[L200, {___, a___, b___, c___, b___, a___, ___} /; Length[{c}] > 10]];
```

```
(* ---- 2. Case M: distinct equal-sum contiguous blocks, N=60 (perf-doc M) - *)
LM60 = Table[Mod[k, 4] + 1, {k, 60}];
bench["Case M equal-sum blocks, N=60",
  ReplaceList[LM60, {___, b1___ /; Total[{b1}] == 5, b2___ /; Total[{b2}] == 5,
    b3___ /; Total[{b3}] == 5, ___} :> {b1, b2, b3}];];
check["Case M equal-sum blocks, N=60",
  Length[ReplaceList[LM60, {___, b1___ /; Total[{b1}] == 5,
    b2___ /; Total[{b2}] == 5, b3___ /; Total[{b3}] == 5, ___} :> {b1, b2, b3}]]];

(* ---- 3. Case M at N=80: the scaling axis (benchOnce -- WL is seconds) ---- *)
LM80 = Table[Mod[k, 4] + 1, {k, 80}];
benchOnce["Case M equal-sum blocks, N=80",
  ReplaceList[LM80, {___, b1___ /; Total[{b1}] == 5, b2___ /; Total[{b2}] == 5,
    b3___ /; Total[{b3}] == 5, ___} :> {b1, b2, b3}];];
check["Case M equal-sum blocks, N=80",
  Length[ReplaceList[LM80, {___, b1___ /; Total[{b1}] == 5,
    b2___ /; Total[{b2}] == 5, b3___ /; Total[{b3}] == 5, ___} :> {b1, b2, b3}]]];
```

A.3 Experiment 42 — rewrite (42-eval-rewrite/eval_rewrite.m)

ReplaceRepeated (//.) — matcher + replace_bindings + the fixed-point convergence check, once per pass over many passes. The pairwise-fold row is the matcher-bound regression sentinel tracked by bench_eval.c.

```
Get["../harness.m"];

(* ---- 1. Bubble sort by adjacent-swap rewrite ----- *)
rev30 = Reverse[Range[30]];
bench["bubble sort //. , 30 elts",
  rev30 //. {p___, u_, v_, q___} /; u > v :> {p, v, u, q}];];
check["bubble sort //. , 30 elts",
  (rev30 //. {p___, u_, v_, q___} /; u > v :> {p, v, u, q}) === Range[30]];

(* ---- 2. Pairwise left-fold to a scalar (bench_eval sentinel) ----- *)
bench["pairwise fold //. , Range[400]",
  Range[400] //. {a_, b_, r___} :> {a + b, r}];];
check["pairwise fold //. , Range[400]",
  First[Range[400] //. {a_, b_, r___} :> {a + b, r}]]];

(* ---- 3. Same fold at 800: the O(n^2) rewrite axis ----- *)
bench["pairwise fold //. , Range[800]",
  Range[800] //. {a_, b_, r___} :> {a + b, r}];];
check["pairwise fold //. , Range[800]",
  First[Range[800] //. {a_, b_, r___} :> {a + b, r}]]];

(* ---- 4. Peano addition by structural rewrite (purely symbolic) ----- *)
Clear[pl, sx];
peanoRules = {pl[0, n_] :> n, pl[sx[m_], n_] :> sx[pl[m, n]]};
n1 = Nest[sx, 0, 60]; n2 = Nest[sx, 0, 60];
bench["Peano add via //. , 60+60", pl[n1, n2] //. peanoRules];];
check["Peano add via //. , 60+60", (pl[n1, n2] //. peanoRules) === Nest[sx, 0, 120]]];
```

A.4 Experiment 43 — arithmetic (43-eval-arithmetic/eval_arithmetic.m)

The interpreter's per-attempt constant on ordinary Length/Plus/Times/Equal plus the Orderless canonicalization of a large symbolic sum. Case B walks the identical search space in both engines, isolating the per-attempt

evaluator constant; the Rational total ($k/3$) never packs.

```
Get["../harness.m"];

(* ---- 1. Case B: typed sequence blanks + linear length guard ----- *)
(* 580 guard evaluations; the matcher walks the identical search space as WL,
   so the difference is purely the per-attempt evaluator constant. *)
r500 = Range[500];
bench["Case B typed, |x|+|y|+|z| guard",
  MatchQ[r500, {xx : Repeated[_Integer], yy : Repeated[_Integer],
    zz : Repeated[_Integer]} /; 3 Length[{xx}] + 5 Length[{yy]} == Length[{zz}]]];
check["Case B typed, |x|+|y|+|z| guard",
  MatchQ[r500, {xx : Repeated[_Integer], yy : Repeated[_Integer],
    zz : Repeated[_Integer]} /; 3 Length[{xx}] + 5 Length[{yy]} == Length[{zz}]]];

(* ---- 2. Case B untyped: isolates the per-element _Integer re-check cost -- *)
bench["Case B untyped __, same guard",
  MatchQ[r500, {xx_, yy_, zz_} /;
    3 Length[{xx}] + 5 Length[{yy]} == Length[{zz}]]];
check["Case B untyped __, same guard",
  MatchQ[r500, {xx_, yy_, zz_} /;
    3 Length[{xx}] + 5 Length[{yy]} == Length[{zz}]]];

(* ---- 3. Orderless canonicalization of a 2000-term symbolic sum ----- *)
bench["Length[Plus @@ 2000 symbolic terms]",
  Length[Plus @@ Table[c[k] x, {k, 2000}]]];
check["Length[Plus @@ 2000 symbolic terms]",
  Length[Plus @@ Table[c[k] x, {k, 2000}]]];

(* ---- 4. Rational accumulation, 20000 terms (never packs) ----- *)
bench["Total[Table[k/3, 20000]]", Total[Table[k/3, {k, 20000}]]];
check["Total[Table[k/3, 20000]]", Total[Table[k/3, {k, 20000}]]];

(* ---- 5. Nested symbolic Plus collapse, 4000 deep ----- *)
bench["Nest[#+a&, x, 4000] then a->0", Nest[# + a &, x, 4000] /. a -> 0];
check["Nest[#+a&, x, 4000] then a->0", (Nest[# + a &, x, 4000] /. a -> 0) === x];
```

A.5 Experiment 44 — structural (44-eval-structural/eval_structural.m)

The generic per-element dispatch loops behind Map/Fold/Count/Position over lists of symbolic `ff[k]` nodes. A symbolic head can never pack, so every element is dispatched through the evaluator + matcher.

```
Get["../harness.m"];

Clear[ff, sq, fp];
sq[ff[u_]] := u^2; (* Map body: one DownValue per element *)
fp[acc_, ff[k_]] := acc + k; (* Fold body: pattern-bind each element *)

S5 = Table[ff[k], {k, 5000}]; (* symbolic -> never packs *)

(* ---- 1. Map a DownValue over 5000 symbolic nodes ----- *)
bench["Map[sq, 5000 f[k] nodes]", Map[sq, S5]];
check["Map[sq, 5000 f[k] nodes]", LeafCount[Map[sq, S5]]];

(* ---- 2. Fold a two-arg pattern over 5000 nodes ----- *)
bench["Fold[fp, 0, 5000 f[k] nodes]", Fold[fp, 0, S5]];
check["Fold[fp, 0, 5000 f[k] nodes]", Fold[fp, 0, S5]];
```

```

(* ---- 3. Count with a PatternTest predicate ----- *)
bench["Count[5000 nodes, f[_?PrimeQ]]", Count[S5, ff[_?PrimeQ]]];
check["Count[5000 nodes, f[_?PrimeQ]]", Count[S5, ff[_?PrimeQ]]];

(* ---- 4. Position with a PatternTest predicate ----- *)
S4 = Table[ff[k], {k, 4000}];
bench["Position[4000 nodes, f[_?EvenQ]]", Position[S4, ff[_?EvenQ]]];
check["Position[4000 nodes, f[_?EvenQ]]", Length[Position[S4, ff[_?EvenQ]]]];

```

A.6 Experiment 45 — construction (45-eval-construction/eval_construction.m)

Node allocation and whole-tree traversal/rebuild on distinct-node symbolic structures (distinct so copy-on-write refcounting cannot collapse them to a shared DAG and measure sharing instead of construction).

```

Get["../harness.m"];

Clear[node, gg];
wide = Table[node[k, k^2, k^3], {k, 15000}]; (* built once, outside timing *)

(* ---- 1. Construct 15000 distinct symbolic nodes ----- *)
bench["build 15000 node[k,k^2,k^3]", Table[node[k, k^2, k^3], {k, 15000}]];
check["build 15000 node[k,k^2,k^3]", LeafCount[Table[node[k, k^2, k^3], {k, 15000}]]];

(* ---- 2. Full-tree leaf traversal ----- *)
bench["LeafCount over 60001 leaves", LeafCount[wide]];
check["LeafCount over 60001 leaves", LeafCount[wide]];

(* ---- 3. Whole-tree rewrite/rebuild via ReplaceAll ----- *)
bench["ReplaceAll rebuild, 15000 nodes", wide /. node[a_, b_, c_] :> a + b + c];
check["ReplaceAll rebuild, 15000 nodes", Total[wide /. node[a_, b_, c_] :> a + b + c]];

(* ---- 4. Deep single-arg nest: 900-level traversal ----- *)
bench["Depth[Nest[gg, x0, 900]]", Depth[Nest[gg, x0, 900]]];
check["Depth[Nest[gg, x0, 900]]", Depth[Nest[gg, x0, 900]]];

```

A.7 Experiment 46 — control-flow (46-eval-controlflow/eval_controlflow.m)

Tight interpreted loops with small bodies — Do/If/Set, guarded DownValues, and Rational FixedPoint iteration. With a trivial body, the per-iteration interpreter constant is the whole cost. Every loop keeps a scalar accumulator; nothing packs or compiles.

```

Get["../harness.m"];

(* ---- 1. Do loop with a symbolic Set accumulator (100k iterations) ----- *)
(* Result is 100000 a; hammers the Set OwnValue fast path + Plus each step. *)
bench["Do[s=s+a, 100000]", Module[{s = 0}, Do[s = s + a, {100000}]; s];
check["Do[s=s+a, 100000]", (Module[{s = 0}, Do[s = s + a, {100000}]; s) /. a -> 1];

(* ---- 2. Do + If + PrimeQ over 10000 integers ----- *)
bench["Do[If[PrimeQ,...], 10000]",
  Module[{s = 0}, Do[If[PrimeQ[k], s = s + k], {k, 10000}]; s];
check["Do[If[PrimeQ,...], 10000]",
  Module[{s = 0}, Do[If[PrimeQ[k], s = s + k], {k, 10000}]; s];

(* ---- 3. Newton's method on exact Rationals (6 steps) ----- *)

```



```

bench["FixedPoint Newton sqrt2, Rational",
  FixedPoint[Together[# - (#^2 - 2)/(2 #)] &, 1, 6];];
check["FixedPoint Newton sqrt2, Rational",
  FixedPoint[Together[# - (#^2 - 2)/(2 #)] &, 1, 6];];

(* ---- 4. Collatz length via guarded recursive DownValues ----- *)
Clear[col];
col[1] := 0;
col[n_ /; EvenQ[n]] := 1 + col[n/2];
col[n_ /; OddQ[n]] := 1 + col[3 n + 1];
bench["Collatz col[27] guarded rules", col[27];];
check["Collatz col[27] guarded rules", col[27];];

```

The five experiments below are the Extended stress tests (2026-08-12), measured at Mathilda 0.039. Same harness, same one-source-run-on-both-engines contract.

A.8 Experiment 47 — memoization (**47-eval-memoization/eval_memoization.m**)

Building and dispatching through a DownValue table that grows to hundreds or thousands of entries. Each timed body clears and rebuilds its table so every rep pays the full construction — a memo table persists across the harness's warm-up + 3 reps, which would otherwise reduce reps 2-4 to O(1) cached lookups.

```

Get["../harness.m"];

$RecursionLimit = 100000;

(* ---- 1. Memoized Fibonacci: 1000 DownValues, built then looked up ----- *)
bench["memo fib[1000]",
  (Clear[fibm]; fibm[0] = 0; fibm[1] = 1;
   fibm[n_] := fibm[n] = fibm[n - 1] + fibm[n - 2]; fibm[1000])];
check["memo fib[1000]",
  (Clear[fibm]; fibm[0] = 0; fibm[1] = 1;
   fibm[n_] := fibm[n] = fibm[n - 1] + fibm[n - 2]; fibm[1000])];

(* ---- 2. Memoized Pascal recurrence: 2-arg memo, ~2500 entries ----- *)
bench["memo binom[100,50]",
  (Clear[bnm]; bnm[n_, 0] := 1; bnm[n_, n_] := 1;
   bnm[n_, k_] := bnm[n, k] = bnm[n - 1, k - 1] + bnm[n - 1, k]; bnm[100, 50])];
check["memo binom[100,50]",
  (Clear[bnm]; bnm[n_, 0] := 1; bnm[n_, n_] := 1;
   bnm[n_, k_] := bnm[n, k] = bnm[n - 1, k - 1] + bnm[n - 1, k]; bnm[100, 50])];

(* ---- 3. 5000 flat facts f[k]=k^2, then sum them (dispatch index at scale) *)
bench["build 5000 facts, sum",
  (Clear[fct]; Do[fct[k] = k^2, {k, 5000}]; Sum[fct[k], {k, 5000}])];
check["build 5000 facts, sum",
  (Clear[fct]; Do[fct[k] = k^2, {k, 5000}]; Sum[fct[k], {k, 5000}])];

```

A.9 Experiment 48 — scoping (**48-eval-scoping/eval_scoping.m**)

Module (fresh renamed locals per instantiation), With (constant substitution), and pure-function / Slot application — constructs that create and tear down bindings on every call, with no native kernel to divert to. All operands are symbolic or scalar, so nothing packs.

```

Get["../harness.m"];

$RecursionLimit = 100000;

```



```

(* ---- 1. Module-bodied recursion: fresh locals at every frame ----- *)
Clear[gs]; gs[0] := 0; gs[n_] := Module[{p}, p = gs[n - 1]; p + n];
bench["Module recursion gs[1500]", gs[1500]];
check["Module recursion gs[1500]", gs[1500]];

(* ---- 2. With substitution inside a 30000-iteration loop ----- *)
bench["With loop, 30000",
  Module[{s = 0}, Do[With[{u = k}, s = s + u^2], {k, 30000}]; s];];
check["With loop, 30000",
  Module[{s = 0}, Do[With[{u = k}, s = s + u^2], {k, 30000}]; s];];

(* ---- 3. Module symbol generation: 30000 fresh lexical scopes ----- *)
bench["Module gen, 30000 scopes",
  LeafCount[Table[Module[{a, b, c}, a + b + c], {30000}]]];];
check["Module gen, 30000 scopes",
  LeafCount[Table[Module[{a, b, c}, a + b + c], {30000}]]];];

(* ---- 4. Pure-function (Slot) application folded 6000 deep (symbolic) ---- *)
bench["Fold pure-function, 6000",
  Fold[(cs[#1, #2]) &, x0, Table[a[k], {k, 6000}]]];];
check["Fold pure-function, 6000",
  Depth[Fold[(cs[#1, #2]) &, x0, Table[a[k], {k, 6000}]]]]];];

```

A.10 Experiment 49 — higher-order (49-eval-higher-order/eval_higher_order.m)

Outer / Sort / FoldList / Apply — operators that build or reshape large expression trees element by element through the generic evaluator. Every operand carries a symbolic head, so nothing packs; Sort exercises the canonical-order comparator over 3000 terms (its check is the order-independent `Total[Sort[l]] == Total[l]`, since the two engines disagree on symbolic order).

```
Get["../harness.m"];
```

```
symL = Table[cf[Mod[k^2, 101]] q^Mod[k, 7], {k, 3000}]; (* symbolic -> never packs *)
```

```

(* ---- 1. Outer product: 150x150 = 22500 symbolic nodes ----- *)
bench["Outer 150x150 symbolic",
  Outer[gg, Table[aa[i], {i, 150}], Table[bb[j], {j, 150}]]];];
check["Outer 150x150 symbolic",
  LeafCount[Outer[gg, Table[aa[i], {i, 150}], Table[bb[j], {j, 150}]]]]];];

(* ---- 2. Sort 3000 symbolic terms (canonical-order comparator) ----- *)
bench["Sort 3000 symbolic", Sort[symL]];];
check["Sort 3000 symbolic", Total[Sort[symL]] == Total[symL]];];

(* ---- 3. FoldList: build 6000 nested symbolic accumulations ----- *)
bench["FoldList 6000 symbolic", FoldList[hh, x0, Table[cc[k], {k, 6000}]]];];
check["FoldList 6000 symbolic", Depth[FoldList[hh, x0, Table[cc[k], {k, 6000}]]]]];];

(* ---- 4. Apply at level 1: restructure 5000 triples ----- *)
bench["Apply Plus @ 5000 triples",
  Apply[Plus, Table[{aa[k], bb[k], cc[k]}, {k, 5000}], {1}]]];];
check["Apply Plus @ 5000 triples",
  LeafCount[Apply[Plus, Table[{aa[k], bb[k], cc[k]}, {k, 5000}], {1}]]]]];];

```

A.11 Experiment 50 — dispatch-extreme (50-eval-dispatch-extreme/eval_dispatch_extreme.m)

The same machinery as experiments 40-42 pushed well past their sizes — naive fib[31], ack[3,6], the $O(n^2)$ fold at Range[1200], and the equal-sum partition search at N=90 — to confirm the per-attempt constant and the backtracking advantage hold under stress. The two heaviest single runs use benchOnce.

```
Get["../harness.m"];

$RecursionLimit = 100000;

(* ---- 1. Naive Fibonacci at 31 (single timed run) ----- *)
Clear[fibn]; fibn[0] = 0; fibn[1] = 1; fibn[n_] := fibn[n - 1] + fibn[n - 2];
benchOnce["naive fib[31]", fibn[31]];
check["naive fib[31]", fibn[31]];

(* ---- 2. Ackermann ack[3,6] (harder than ack[3,4]) ----- *)
Clear[ackn]; ackn[0, n_] := n + 1; ackn[m_, 0] := ackn[m - 1, 1];
ackn[m_, n_] := ackn[m - 1, ackn[m, n - 1]];
bench["ack[3,6] nested recursion", ackn[3, 6]];
check["ack[3,6] nested recursion", ackn[3, 6]];

(* ---- 3. Pairwise left-fold //. at Range[1200] ( $O(n^2)$  rewrite axis) ----- *)
bench["pairwise fold //. , Range[1200]",
  Range[1200] //. {a_, b_, r___} := {a + b, r};];
check["pairwise fold //. , Range[1200]",
  First[Range[1200] //. {a_, b_, r___} := {a + b, r}]];

(* ---- 4. Case M equal-sum blocks at N=90 (backtracking; single run) ----- *)
LM90 = Table[Mod[k, 4] + 1, {k, 90}];
benchOnce["Case M equal-sum blocks, N=90",
  ReplaceList[LM90, {___, b1___ /; Total[{b1}] == 5, b2___ /; Total[{b2}] == 5,
    b3___ /; Total[{b3}] == 5, ___} := {b1, b2, b3}];];
check["Case M equal-sum blocks, N=90",
  Length[ReplaceList[LM90, {___, b1___ /; Total[{b1}] == 5,
    b2___ /; Total[{b2}] == 5, b3___ /; Total[{b3}] == 5, ___} := {b1, b2, b3}]]];
```

A.12 Experiment 51 — arithmetic-extreme (51-eval-arithmetic-extreme/eval_arithmetic_extreme.m)

Experiments 43 and 46 scaled up, plus a bignum axis: a 5000-term Orderless sum, a 300000-iteration Do loop, a 60000-term Rational total, and 12000! evaluated as 12000 successive Times through the evaluation loop. Symbolic terms and Rationals never pack; the factorial keeps a scalar bignum accumulator.

```
Get["../harness.m"];

(* ---- 1. Orderless canonicalization of a 5000-term symbolic sum ----- *)
bench["Length[Plus @@ 5000 symbolic terms]",
  Length[Plus @@ Table[cc[k] x, {k, 5000}]]];
check["Length[Plus @@ 5000 symbolic terms]",
  Length[Plus @@ Table[cc[k] x, {k, 5000}]]];

(* ---- 2. Do loop with a symbolic Set accumulator, 300000 iterations ----- *)
bench["Do[s=s+a, 300000]", Module[{s = 0}, Do[s = s + a, {300000}]; s];];
check["Do[s=s+a, 300000]", (Module[{s = 0}, Do[s = s + a, {300000}]; s)] /. a -> 1];

(* ---- 3. Rational accumulation, 60000 terms (never packs) ----- *)
bench["Total[Table[k/3, 60000]]", Total[Table[k/3, {k, 60000}]]];
check["Total[Table[k/3, 60000]]", Total[Table[k/3, {k, 60000}]]];
```

```
(* ---- 4. 12000! via Fold[Times]: 12000 bignum multiplies through the loop - *)
bench["Fold[Times, 1, Range[12000]]", Fold[Times, 1, Range[12000]]];
check["Fold[Times, 1, Range[12000]]", Fold[Times, 1, Range[12000]] == 12000!];
```

A.13 Experiment 52 — Apply / Map (52-eval-apply-map/apply_map.m)

The dedicated head-to-head for the 2026-08-12 Apply/Map fast paths (results in Apply / Map restructuring vs WolframKernel 14.0 above). Unlike the experiments above it does not use the shared Print["BENCH"...] harness: it is run directly against both kernels — ./Mathilda -file apply_map.m and WolframKernel -noprompt -script apply_map.m — and emits bare integer-microsecond timings (in order) then bare check values (in order), the one output form that parses cleanly out of both. The lists are built once, outside timing, so each case measures only the restructuring.

```
$Reps = 5;
SetAttributes[bench, HoldAll];
bench[e_] := Module[{ts}, e; ts = Table[First[AbsoluteTiming[e]], {$Reps}];
  Round[1.*^6 Min[ts]]];

symP = Table[c[k] x, {k, 5000}]; (* 5000 symbolic terms c[k] x *)
tri = Table[{aa[k], bb[k], cc[k]}, {k, 5000}]; (* 5000 triples *)
comp = Table[gg[aa[k], bb[k], cc[k]], {k, 5000}]; (* compound elements (depth 2) *)
deep = Table[gg[hh[aa[k], bb[k]], hh[cc[k], dd[k]]], {k, 5000}]; (* deep elements (depth 3) *)

(* ---- timings (integer microseconds, in order) ---- *)
Print[bench[ff @@ symP]]; (* 1. Apply @@ (level 0) *)
Print[bench[ff @@@ tri]]; (* 2. MapApply @@@ (level 1) *)
Print[bench[ff /@ comp]]; (* 3. Map /@ compound (level 1) *)
Print[bench[ff /@ deep]]; (* 4. Map /@ deep (level 1) *)
Print[bench[Map[ff, comp, {0}]]]; (* 5. Map at level 0 *)
Print[bench[Scan[ff, comp]]]; (* 6. Scan compound (level 1) *)
Print[bench[Scan[ff, deep]]]; (* 7. Scan deep (level 1) *)
Print[bench[MapIndexed[ff, comp]]]; (* 8. MapIndexed compound (level 1) *)
Print[bench[MapIndexed[ff, deep]]]; (* 9. MapIndexed deep (level 1) *)
Print[bench[Apply[Plus, tri, {1}]]]; (* 10. MapApply Plus (level 1) *)
Print[bench[Length[Plus @@ symP]]]; (* 11. Plus @@ grouping (5000 terms) *)

(* ---- checks (bare values, in order; must agree across engines) ---- *)
Print[Length[ff @@ symP]];
Print[LeafCount[ff @@@ tri]];
Print[LeafCount[ff /@ comp]];
Print[LeafCount[ff /@ deep]];
Print[LeafCount[Map[ff, comp, {0}]]];
Print[Module[{s = 0}, Scan[(s = s + 1) &, comp]; s]];
Print[Module[{s = 0}, Scan[(s = s + 1) &, deep]; s]];
Print[LeafCount[MapIndexed[ff, comp]]];
Print[LeafCount[MapIndexed[ff, deep]]];
Print[LeafCount[Apply[Plus, tri, {1}]]];
Print[Length[Plus @@ symP]];
```